GameScope

Student: Yousef Malik **Course:** IT401
 Assignment: A3 - Database Integration, Persistence, and CRUD
Application name: GameScope

Project Overview

GameScope is a Flask web-intelligence application for discovering PC games on Steam. It preserves the searchable 24-game local collection created for A1 and adds a live Hidden Gems experience. A3 adds a persistent SQLAlchemy-backed library for information acquired from Steam 250, with web-based create, read, update, and delete operations. The interface also combines authenticated RAWG metadata, structured information scraped from Steam 250, and current player reviews from Steam.

GameScope focuses on games that players rate highly but that have not yet reached the review volume of major releases. Users can search by game title or filter the scraped ranking by genre without changing the source code. Selecting a result displays current metadata and representative player-review excerpts.

What "hidden gem" means in this project

GameScope does not invent a hidden-gem label or score. It uses the ordering and score published by Steam 250's Hidden Gems chart. That chart emphasizes games with strong player approval and comparatively limited review exposure. The ranking is a discovery signal rather than a guarantee that every user will like every result.

Application pages

Route	Purpose	Data used
`/`	Introduces GameScope and links to both discovery tools	Static application content
`/explore`	Searches and filters the original A1 collection	`data/games.json`
`/hidden-gems`	Displays up to 250 Steam 250 records and enriches a search or selected game	Steam 250, RAWG, and Steam Reviews
`/library`	Searches and manages permanently stored records	MySQL, or local SQLite when MySQL settings are not configured

External Information Sources

RAWG Video Games Database API

- Service: RAWG API
- Endpoint: `GET https://api.rawg.io/api/games`
- Authentication: `RAWG_API_KEY` query parameter, loaded from the environment
- User parameter: the title entered in the Hidden Gems search form
- Fields used: title, release date, rating, rating count, Metacritic score,

average playtime, genres, platforms, image, and source URL

- Purpose: enrich a Steam hidden-gem candidate with normalized PC game metadata

RAWG requires attribution and a link on pages where its information appears. GameScope displays this attribution beside every API result.

Steam 250 webpage

- Source: Steam 250 Hidden Gems
- Official mirror: Steam 250 generated HTML on GitHub
- Acquisition method: `requests` and `BeautifulSoup`
- Fields extracted for each record: rank, title, Steam App ID, primary tag,

release year, Steam 250 score, review count, approval percentage, price, image URL, and Steam Store URL

- Purpose: supply a current ranking of well-reviewed games with relatively low

visibility

The scraper requests one public ranking page, identifies itself with a custom user-agent, caches successful results for 15 minutes, and does not crawl linked pages.

Steam User Reviews endpoint

- Documentation: Steam User Reviews
- Endpoint: `GET https://store.steampowered.com/appreviews/<appid>?json=1`
- Fields used: review description, positive/negative totals, written review,

recommendation, playtime, helpful votes, comment count, and developer response

- Purpose: add current, detailed player experiences to the selected result

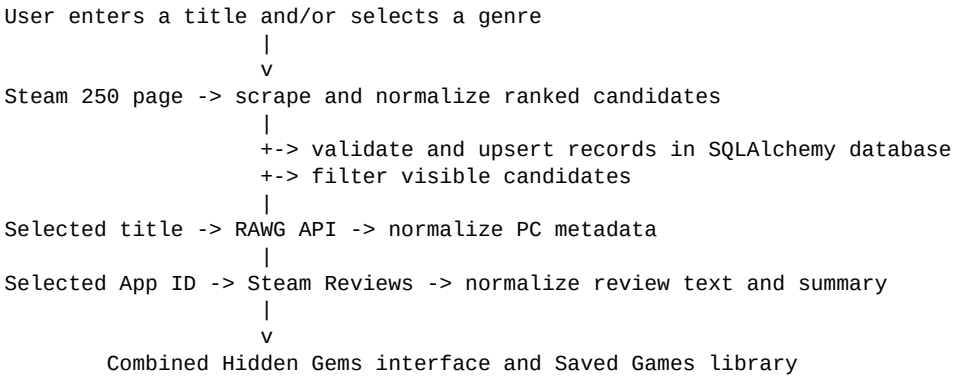
Steam's documented endpoint returns JSON and is treated as a supporting API, not as the assignment's HTML scraper.

API and Web Scraping Differences

An API provides a documented machine-readable interface. GameScope sends parameters to RAWG and Steam and processes their JSON responses using known field names. Authentication, response codes, and rate limits are part of the API contract.

Web scraping retrieves a webpage intended primarily for people and extracts selected values from its HTML. GameScope uses BeautifulSoup to locate Steam 250 game cards and clean their visible text into dictionaries. Scraping is more fragile because CSS classes and page structure can change without notice, so the scraper checks for missing elements and reports a safe error instead of crashing.

Application Workflow



The local `/explore` route remains separate and continues to read `data/games.json`.

Detailed request lifecycle

1 A request to `/hidden-gems` asks the scraper for as many as 250 ranked

records.

1 The scraper tries the Steam 250 website first, then Steam 250's official

generated HTML on GitHub, and finally the saved local snapshot. The interface shows a notice whenever the primary source was not used.

1 BeautifulSoup converts every usable HTML card into a consistently shaped

dictionary. Incomplete cards are skipped and optional values receive safe defaults.

1 A selected genre filters the records by their primary Steam 250 tag. A title

or keyword search filters the visible records by title or tag.

1 When a title is submitted, the same text is sent to RAWG as the `search`

parameter. RAWG is limited to PC results and the first close match is normalized for display.

1 Choosing **View Intelligence** submits the game's numeric Steam App ID. The

Steam Reviews endpoint returns the current review summary and representative English-language review excerpts.

1 Valid Steam 250 records are inserted or refreshed in the `game_records`

table. Stable source URLs and Steam App IDs prevent repeated requests from producing duplicate entries.

1 Flask renders the scraped ranking, API metadata, review information,

attribution links, and any source-specific warning on one page. `/library` reads saved records back from the database for search and CRUD operations.

Source priority, caching, and data freshness

```
Steam250.com
     | unavailable or invalid HTML
     v
Official Steam 250 GitHub mirror
     | unavailable or invalid HTML
     v
data/steam250_fallback.json (saved 250-record snapshot)
```

A successful scrape is cached in process memory for 900 seconds (15 minutes) to reduce repeated traffic. This cache is not shared between server processes and is cleared on restart. Separately, normalized Steam 250 records are stored permanently in `game_records`. RAWG searches, Steam review responses, and the 24-game A1 JSON collection remain request-time or file-backed information and are not copied into the database.

Information Model

Local game

- `id`, `title`, `genre`, `mode`, `price`, `rating`
- `estimated_playtime`, `release_year`, `description`, `tags`

Scraped hidden-gem candidate

- `rank`, `title`, `steam_app_id`, `tag`, `year`
- `score`, `review_count`, `approval`, `price`
- `image`, `steam_url`

RAWG API game

- `id`, `title`, `released`, `rating`, `ratings_count`, `metacritic`
- `playtime`, `genres`, `platforms`, `image`, `source_url`

Steam review summary and review

- Summary: `score_description`, `total_positive`, `total_negative`,

`total_reviews`

- Review: `text`, `recommended`, `helpful_votes`, `comment_count`,

`playtime_hours`, `developer_response`

Persistent database record

The `game_records` table stores each managed or acquired record with `id`, `title`, `description`, `source`, `source_url`, `retrieved_date`, and `created_date`. Optional Steam 250 attributes are `steam_app_id`, `rank`, `tag`, `release_year`, `steam250_score`, `review_count`, `approval`, `price`, and `image_url`.

Local records are fictional demonstration data stored in the repository. Steam 250 ranking data is acquired at request time and refreshed in the database; the ranking snapshot remains a resilience source. RAWG and Steam reviews are displayed for the request and are not persisted.

Application Architecture

GameScope uses a small Flask service structure so routing, external data access, and presentation remain separate:

```
app.py                          Flask application factory, database setup, and CLI command
config.py                       Environment variables, URLs, timeouts, and cache settings
routes/main.py                  Discovery routes, filtering, and service coordination
routes/library.py               Database-backed library CRUD routes and form validation
models/__init__.py               Flask-SQLAlchemy extension and GameRecord schema
services/persistence_service.py Maps/upserts acquired Steam 250 records
services/rawg_service.py        Authenticated RAWG request and JSON normalization
services/steam250_scraper.py    HTML fetching, parsing, retry, cache, and fallback logic
services/steam_review_service.py Steam review request and JSON normalization
services/external_errors.py     Safe application-facing external service errors
templates/                      Jinja page templates
static/                         Shared CSS, logo, and report screenshots
data/games.json                 Original 24-game A1 collection
data/steam250_fallback.json     Saved 250-record resilience snapshot
scripts/refresh_steam250_snapshot.py Snapshot maintenance command
tests/test_app.py               Automated route, parser, API, and failure tests
```

`app.py` selects a configuration, initializes Flask-SQLAlchemy, registers routes, and exposes `flask --app app init-db` for initial table creation. `routes/main.py` coordinates discovery and acquired-record persistence; `routes/library.py` validates CRUD form input. External services normalize their responses, and expected source failures are presented as readable notices without preventing unaffected data from being displayed.

Environment Variables

Variable	Required	Description
`RAWG_API_KEY`	Yes for RAWG search	Private key issued by RAWG
`SECRET_KEY`	Recommended	Flask session/security secret
`DATABASE_URL`	Optional	SQLAlchemy connection URL; use the CI-Keys MySQL URL when supplied
`MYSQL_HOST`	Optional	MySQL server hostname, when not using `DATABASE_URL`
`MYSQL_PORT`	Optional	MySQL port; defaults to `3306`
`MYSQL_USER`	Optional	MySQL username
`MYSQL_PASSWORD`	Optional	MySQL password
`MYSQL_DATABASE`	Optional	Existing MySQL schema/database name
`FLASK_ENV`	No	`development` or `production`

Set either `DATABASE_URL` or the `MYSQL_HOST`, `MYSQL_USER`, `MYSQL_PASSWORD`, and `MYSQL_DATABASE` values to use MySQL. The application selects the `mysql+pymysql` driver for MySQL. If no database settings are present, it uses a persistent SQLite file in Flask's instance directory for local development and automated tests. The real `.env` file is excluded through `.gitignore`; `.env.example` contains variable names only.

Installation Instructions

1 Clone the repository and enter its directory.

``bash git clone <repository-url> cd MalikYousef-it401-final-project-GameScope-main ``

1 Create and activate a virtual environment.

``bash python -m venv .venv # Windows PowerShell .venv\Scripts\Activate.ps1 ``

1 Install dependencies.

``bash python -m pip install -r requirements.txt ``

1 Register for a free key at RAWG.

1 Copy `.env.example` to `.env` and add the private key.

``dotenv RAWG_API_KEY=your_private_key_here SECRET_KEY=your_random_secret_here DATABASE_URL=mysql+pymysql://username:password@host:3306/database?charset=utf8mb4 ``

Use the MySQL connection values provided through CI-Keys. URL-encode special characters in a `DATABASE_URL`, or provide the `MYSQL_*` settings separately. The database/schema must already exist and the account must be allowed to create tables. Leave `DATABASE_URL` empty to use local SQLite instead.

1 Initialize the tables.

``bash flask --app app init-db ``

1 Start the application.

``bash python app.py ``

1 Open http://127.0.0.1:5000.

If PowerShell prevents activation of the virtual environment, the application can still be run by addressing its Python executable directly:

```
.venv\Scripts\python.exe -m pip install -r requirements.txt
.venv\Scripts\python.exe app.py
```

Do not place a key directly in `rawg_service.py`, `api_service.py`, or another Python file. Restart the Flask server after changing `.env`, because environment values are read during application startup.

Using the Application

Browse the local collection

1 Open **Local Collection**, or visit `/explore`.

2 Enter a keyword or choose a genre, play mode, and minimum rating.

3 Submit the form. Matching local games are sorted from highest to lowest

rating.

1 Clear the form to restore all 24 demonstration records.

Discover and investigate hidden gems

1 Open **Hidden Gems**, or visit `/hidden-gems`. An unfiltered load displays

up to all 250 ranked records from the active Steam 250 source.

1 Select a genre to filter by the primary tag supplied by Steam 250.

2 Enter a title or keyword to filter the cards and request matching RAWG

metadata. The RAWG information panel appears when a match is returned.

1 Select **View Intelligence** on a card to request its Steam review summary

and detailed review excerpts. The selected App ID is validated before any Steam request is made.

1 Follow the displayed source links to inspect the original information.

Genre filtering is based on the one primary tag available in each Steam 250 record. A game may have additional Steam genres that are not represented by that tag. A search can also return fewer than 250 cards because the filtering is intentionally applied to the complete ranking.

Dependencies

Package	Role
Flask	Routing, configuration, and Jinja template rendering
Flask-SQLAlchemy	ORM models, database sessions, and SQLAlchemy integration
PyMySQL	MySQL DBAPI driver
Requests	HTTP calls to RAWG, Steam 250, its mirror, and Steam Reviews
Beautiful Soup	Parsing and cleaning the Steam 250 HTML
python-dotenv	Loading private configuration from `.env`
pytest	Automated application and service tests

Exact minimum versions are listed in `requirements.txt`.

Maintaining the Steam 250 Snapshot

The checked-in snapshot keeps the assignment demonstrable during a Steam 250 DNS or network outage. Refresh it only when one of the official HTML sources is available:

```
python scripts/refresh_steam250_snapshot.py
```

The command uses the same parser as the web application, requests up to 250 records, writes formatted JSON to `data/steam250_fallback.json`, and reports the source and number of saved records. Review the resulting Git diff before committing it. The snapshot is public ranking data; it must never contain an API key or other secret.

Current Features

- Responsive Flask homepage and shared navigation
- Searchable and filterable 24-game local collection from A1
- Live Steam 250 Hidden Gems HTML scraper
- Structured extraction and cleaning of multiple game records
- Title search sent as a parameter to the authenticated RAWG API
- PC metadata normalization from RAWG JSON
- Current Steam review summaries and detailed review excerpts
- Genre filtering and title/keyword searching
- Source links and attribution in the interface
- Fifteen-minute scraper cache to limit repeated requests
- A dedicated 30-second scraper timeout for slower Steam 250 responses
- Automatic retries for temporary DNS, connection, and server failures
- Steam 250's official GitHub `gh-pages` mirror as a secondary live HTML source
- A 250-game attributed Steam 250 snapshot used only when the live source is down
- Graceful handling of invalid input and external-source failures
- MySQL persistence for acquired Steam 250 information
- Searchable saved-record library with create, read, update, and delete actions
- Required-field, URL, length, and duplicate validation
- Automated tests that do not require live external services

Error Handling

- **Missing API key:** the page explains that `RAWG_API_KEY` must be added to

`.env`; scraped results remain usable.

- **Invalid input:** title length is limited, unknown genre values are ignored,

and Steam App IDs must contain digits only.

- **API authentication failure:** a safe message asks the user to check the key.
- **Rate limits:** HTTP 429 responses produce a wait-and-retry message.
- **Network failure or timeout:** each source has a timeout and its own readable

notice rather than a Flask error page. Steam 250 falls back to a saved, clearly labeled snapshot so the page remains demonstrable.

- **Empty API results:** the user is told no matching PC game was found.
- **Missing HTML fields:** incomplete Steam 250 records are skipped and optional

fields receive safe defaults.

- **Changed webpage structure:** if no records can be extracted, the page reports

that the format may have changed.

- **Empty reviews:** the page retains the review summary and explains that no

written English reviews were returned.

Ethical Considerations

- GameScope identifies its scraper and makes only one cached request to the

Steam 250 ranking instead of crawling the site.

- Steam 250, RAWG, and Steam are visibly attributed and linked.
- RAWG's free-plan attribution and request limits must be followed.
- Only public player reviews are requested through Steam's documented endpoint.
- GameScope does not collect Steam IDs or attempt to identify reviewers.
- Review excerpts remain the opinions of their authors and should not be treated

as objective facts.

- Site policies and `robots.txt` should be checked again before deploying the

application or increasing request volume.

Security and Privacy

- Secrets are loaded from the ignored `.env` file and are never rendered in a

template or placed in a source URL shown to users.

- `.env.example` contains names and placeholders only, so it is safe to commit.
- Search input is trimmed and length-limited; App IDs must be numeric and genre

values must come from the application's allowlist.

- Jinja auto-escaping remains enabled for values rendered into HTML.
- The application does not use user accounts, persist searches, or collect

reviewer identities.

- The default `SECRET_KEY=dev` fallback is suitable only for local development.

A random value should be supplied through `.env` for deployment.

Known Limitations

- External information requires an internet connection.
- RAWG title matching may choose a similarly named game.
- Steam 250 exposes one primary tag per ranking record, so genre filtering does

not represent every tag a game has on Steam.

- The Steam endpoint provides review text and comment counts but not the full

discussion beneath a review.

- Ranking and price data may change between visits.
- A redesign of Steam 250's HTML may require selector updates.
- The in-memory scraper cache resets when the Flask process restarts.
- The saved 250-record Steam 250 fallback may be older than the live ranking.
- A high approval percentage with few reviews is a discovery signal, not proof

that every player will enjoy the game.

Testing

Run the automated suite with:

```
python -m pytest -q
```

Tests cover the homepage, local filters, HTML extraction, missing credentials, combined API/scraper rendering, and graceful external failure handling. Network responses are mocked so the tests are repeatable and do not consume API limits.

At the time of this update, the suite contains 10 passing tests. Run it again after any route, parser, template, or service change rather than relying on that recorded count.

Troubleshooting

The browser says the page is unreachable

- Confirm that the terminal running `python app.py` is still open and shows a

local address such as `http://127.0.0.1:5000`.

- Use that local address directly; do not substitute a Bing search redirect or

another website URL.

- If port 5000 is already occupied, stop the older Flask process before starting

another instance.

- After editing `.env`, stop the server with `Ctrl+C` and start it again.

A3 Documentation

Project Title

GameScope: Persistent PC Game Discovery
 Application: GameScope
 Student: Yousef Malik
 Course and assignment: IT401, A3 - Database Integration, Persistence, and CRUD

Project Overview

GameScope helps PC players explore a local demonstration catalog and find lesser-known Steam games using Steam 250 rankings. It combines web scraping, RAWG API metadata, and Steam review information. Its intended users are players looking for discovery signals and an administrator or evaluator managing research records through the Saved Games page.

Information Persistence

The application permanently stores Steam 250 Hidden Gems records and records created or edited in Saved Games. Persistence keeps acquired ranking information available across application restarts and supports review and maintenance through CRUD. The A1 collection in `data/games.json`, the 15-minute scraper cache, RAWG search results, and Steam review responses remain file-backed or temporary and are not database records.

Database Schema

The `game_records` table has no relationships to other tables in this version. Each record represents one PC game from Steam 250 or an entry manually managed in the library.

Attribute	Type	Purpose
`id`	Integer, primary key	Stable record identifier
`title`	String(200), required, indexed	Game title and library search field
`description`	Text, required	Summary of the acquired ranking entry or user-provided description

Attribute	Type	Purpose
`source`	String(120), required	Origin name, such as Steam 250
`source_url`	String(500), required, unique	Direct source link and duplicate key
`retrieved_date`	DateTime, required	Most recent acquisition/refresh time
`created_date`	DateTime, required	First database creation time
`steam_app_id`	String(20), optional, unique	Steam identity used to refresh the same game
`rank`, `tag`, `release_year`	Optional ranking fields	Discovery rank and primary classification
`steam250_score`, `review_count`, `approval`, `price`, `image_url`	Optional source attributes	Preserve the useful values acquired from Steam 250

There are no foreign-key relationships because each record is self-contained. Rank, tag, App ID, score, approval, review count, price, year, and image URL are additional fields because they retain useful A2 acquisition attributes for filtering and display without introducing another table.

Application Architecture

`app.py` creates the Flask application, initializes Flask-SQLAlchemy, registers routes, and exposes the `init-db` command. `models/__init__.py` defines the database extension and model. `routes/main.py` coordinates discovery while `routes/library.py` owns library CRUD and form validation. Service modules acquire external data; `services/persistence_service.py` maps and upserts the Steam 250 records. Jinja templates render pages, and `static/style.css` styles them. MySQL is connected through PyMySQL; SQLite is available as a persistent local-development fallback.

Information Flow

1 A browser requests `/hidden-gems` or submits a title/genre filter.

2 The scraper retrieves Steam 250 HTML, its official mirror, or the saved

snapshot, then parses and normalizes usable entries.

1 The persistence service checks required acquisition values and locates

existing entries by source URL or Steam App ID.

1 SQLAlchemy inserts new records or refreshes existing records and commits

them to MySQL (or configured local SQLite).

1 Flask renders the discovery response. The `/library` route reads records

from the database and permits validated create, edit, and delete requests.

1 RAWG and Steam review responses are requested when needed and rendered for

that response only; they are not saved.

Current Features

- Homepage and searchable/filterable A1 catalog
- Steam 250 web scraping with mirror, cache, and saved-snapshot fallback
- RAWG API metadata and Steam review integration
- MySQL-backed persistence for Steam 250 acquired information
- Database-backed Saved Games search, create, read, update, and delete
- Required-field, length, URL-scheme, and duplicate checks
- External-source attribution and readable service/database error feedback

Project Structure

Path	Purpose
`app.py`, `config.py`	Flask factory, database setup command, environment and source configuration
`models/`	SQLAlchemy model and shared database extension
`routes/`	Discovery and CRUD request handlers
`services/`	API clients, scraper, error types, and persistence mapping
`templates/`, `static/`	Jinja views, styles, logo, and A3 screenshots
`data/`	A1 JSON catalog and Steam 250 fallback snapshot
`scripts/`	Snapshot refresh and A3 PDF generation utilities
`tests/`	Isolated route, validation, persistence, and parser tests

Installation Instructions

Clone the repository, create and activate a virtual environment, install `requirements.txt`, copy `.env.example` to `.env`, and configure the RAWG key and CI-Keys MySQL connection variables. Create the target database/schema and grant the account permission to create tables. Then run:

```
flask --app app init-db
python app.py
```

Visit `http://127.0.0.1:5000`. Without MySQL environment variables, the local configuration uses a persistent SQLite database in Flask's `instance/` directory; the MySQL setup is required for the course database deliverable.

Environment Variables

RAWG_API_KEY enables RAWG search; SECRET_KEY configures Flask security. For MySQL, use either DATABASE_URL or MYSQL_HOST, MYSQL_PORT, MYSQL_USER, MYSQL_PASSWORD, and MYSQL_DATABASE. FLASK_ENV optionally selects development or production behavior. No API key, password, or full connection URL belongs in source control or this README.

Error Handling

- **Invalid input:** required fields, length limits, URL format, and allowed

URL schemes are validated before persistence; errors are returned on the form.

- **Missing API key:** RAWG reports the missing RAWG_API_KEY; scraped ranking

results remain available.

- **API/scraper failure:** timeouts, HTTP errors, retries, mirror, and snapshot

fallback are handled by the existing services and surfaced as notices.

- **Duplicates:** the library rejects a repeated source URL; the acquisition

service updates a matching source URL or Steam App ID instead of adding a second row.

- **Database write failure:** failed CRUD commits and acquisition writes are

rolled back and reported; live discovery results can still render if only their persistence attempt fails.

Screenshots

The submission should include captures of the homepage, database-backed Saved Games page, CRUD form, and an example persisted record. These screenshots are not included in this generated report because a local application server was not started for capture. Save the images under static/images/ and add them to this section before final submission; the PDF builder will include referenced images in the next report build.

Design Decisions

- Steam 250 is the persisted A2 dataset because each ranking entry has a stable

game identifier and useful acquisition metadata. RAWG and Steam reviews are deliberately request-scoped and may change frequently.

- One game_records table is sufficient for the current single-entity workflow;

it avoids an unnecessary relationship or advanced repository layer.

- Steam ranking attributes are optional columns so records can also be entered

manually while preserving required title, description, source, and URL.

- source_url is unique and Steam App ID is also unique when present. Manual

duplicates are rejected; repeated acquisition refreshes the existing row.

- Source descriptions are synthesized from rank, tag, and year because Steam

250's listing does not provide a standalone descriptive paragraph.

Known Limitations

- MySQL must be provisioned separately and its connection settings must be

supplied using CI-Keys; the project does not create the MySQL server/schema.

- `db.create_all()` initializes a new schema but does not migrate existing

schemas after model changes; a migration tool may be needed for future updates.

- Steam 250 markup and ranking data can change, and the saved fallback can be

stale. The scraper cache is process-local.

- The app has no authentication or role-based access control, so CRUD is

intended for local/course demonstration rather than public deployment.

- RAWG matching is approximate, external information needs an internet

connection, and the A1 catalog remains a JSON file.

Generating the A3 PDF

After installing dependencies and adding screenshots, regenerate the report with:

```
python scripts/build_a3_report.py
```

The generated `Yousef_Malik_A3_Project_Report.pdf` contains the README's A3 documentation and the referenced screenshots.

Hidden Gems shows a source warning

The application is working as designed when it reports that it used the official mirror or saved snapshot. The warning identifies the data source used for that request. If the snapshot appears instead of live data, verify the computer's internet connection and try again later; Steam 250's hostname may be temporarily unavailable even when other sites work.

Fewer than 250 cards are visible

- Clear the title and genre filters. An unfiltered Hidden Gems request is the

view that can display the full 250-record source list.

- Check the notice at the top of the page to identify whether the primary,

mirror, or snapshot source was used.

- Restart the server to clear its 15-minute in-memory cache if the source was

recently refreshed.

- Run the snapshot refresh command when the local fallback itself needs an

update and an official source is reachable.

RAWG metadata does not appear

- Confirm `.env` contains `RAWG_API_KEY=...` with no placeholder value, then

restart Flask.

- Search for a specific game title. RAWG enrichment happens only when a title

or keyword is submitted on `/hidden-gems`.

- Read the displayed notice for authentication, rate-limit, timeout, network,

or no-match information. Steam 250 results remain available even if RAWG fails.

Reviews do not appear

Use **View Intelligence** on a specific card. Review requests require that card's numeric Steam App ID and are independent of RAWG. Some games may have a summary but no written English excerpt, and Steam outages or rate limits are reported without breaking the rest of the page.

Confirm the project is healthy

From the project directory, run:

```
python -m pytest -q
```

If the tests pass but a live panel fails, the most likely cause is the external service, its credentials, or the network rather than Flask's local routes.

Screenshots

The completed A2 screenshot document is included in the repository: Yousef_Malik_A2_Screenshots.pdf.

It contains labeled evidence of the application's required workflows, including:

1. Homepage with the **Discover Hidden Gems** action
2. Local Collection page showing preserved A1 data
3. Hidden Gems page showing scraped Steam 250 results
4. A title search showing RAWG metadata and its attribution link
5. A selected game showing Steam review summary and detailed review excerpts
6. Missing-key error message (temporarily remove the key for this screenshot)
7. No-results or invalid-search handling
8. Passing automated tests in the terminal

The document also supports the written documentation of the external sources, API and scraping behavior, and credential protection. No private API key is stored in the repository.